RideFare AI

An AI-Powered Taxi Fare Prediction System

Technical Report

Ali Khurram (24K-1000), Mohsin Hassan (24K-1005), Abuzar Mehdi (24K-0630)
RideFare AI Team
April 2026

Python | Flask | scikit-learn | Machine Learning

Executive Summary

RideFare AI is an AI-powered taxi fare prediction platform designed to bring transparency and accuracy to ride-share cost estimation in Pakistan. The system leverages three machine learning regression models — Random Forest, Gradient Boosting, and Linear Regression — trained on a dataset of 10,000 synthesised trip records to predict fares in Pakistani Rupees (PKR).

The project delivers a full-stack solution: a scikit-learn pipeline for model training and evaluation, and a Flask-powered REST API with a dark-themed, responsive web interface. Users can obtain real-time fare predictions, view surge pricing indicators, and compare all three model estimates side by side.

Key Achievements

- Trained three ML models achieving up to $R^2 = 0.998$ (99.8% explained variance)
- Random Forest selected as the best model: MAE = PKR 129, RMSE = PKR 161, $R^2 = 0.998$
- Gradient Boosting achieved equal R^2 with slightly higher error: MAE = PKR 141, RMSE = PKR 187
- Linear Regression established the baseline: MAE = PKR 957, $R^2 = 0.874$
- Five-endpoint Flask REST API serving predictions and model metadata
- Responsive dark-themed web UI with Chart.js visualisations and surge pricing detection
- All models serialised with joblib for fast, sub-millisecond inference

Model	MAE (PKR)	RMSE (PKR)	R^2	Confidence
🥇 **Random Forest**	129	161	0.998	94%
🥈 **Gradient Boosting**	141	187	0.998	92%
🥉 **Linear Regression**	957	1,418	0.874	74%

1. Introduction

1.1 Background

Pakistan's ride-share market has grown substantially with the expansion of platforms such as Uber and Careem in major urban centres including Karachi, Lahore, and Islamabad. Despite this growth, fare transparency remains a significant pain point for passengers. Dynamic surge pricing algorithms are opaque, and riders frequently have no reliable mechanism to estimate trip costs before confirming a booking.

This lack of transparency erodes consumer trust and creates friction in the adoption of digital ride-share services. A data-driven, model-based fare estimator — one that makes its logic explicit — can address this gap directly.

1.2 Problem Statement

Passengers face three core challenges when using ride-share services:

- Unpredictable fares: Surge pricing and opaque algorithms provide no pre-booking estimate.
- No transparency: The contributing factors (distance, traffic, ride type) are invisible to users.
- No comparison tool: No mechanism exists to compare fare estimates across models or service tiers.

1.3 Objectives

This project aims to:

- Build an accurate machine learning fare predictor using multiple regression algorithms.
- Compare the performance of Random Forest, Gradient Boosting, and Linear Regression on the fare prediction task.
- Deploy a full-stack web application exposing the models through a REST API.
- Provide a responsive, user-friendly interface for real-time fare estimation and model comparison.

1.4 Scope

The system operates on a synthetic dataset of 10,000 trip records representing five urban zones. While the dataset is generated algorithmically to reflect real-world fare dynamics, the methodology and architecture are directly applicable to real trip data from ride-share APIs or open datasets.

2. System Architecture

2.1 High-Level Overview

RideFare AI is structured as a three-tier application: a data and model tier, a server-side API tier, and a client-side presentation tier. The architecture follows a standard machine learning serving pattern: models are trained offline and serialised, then loaded at server startup for low-latency inference.

Data Generation → Preprocessing Pipeline → Model Training → Serialisation (joblib) → Flask REST API → Web UI

2.2 Technology Stack

Layer	Technology	Purpose
Data	***pandas, numpy***	Dataset generation, feature engineering, train/test split
ML	***scikit-learn***	Preprocessing pipeline, model training, evaluation metrics
Serialisation	***joblib***	Model persistence (.pkl files) for fast loading
API	***Flask (Python)***	REST endpoints exposing model predictions
Frontend	***HTML5, CSS3, JS***	Dark-themed responsive web interface
Visualisation	***Chart.js 4.4.1***	Real-time bar and gauge charts in the browser

2.3 Component Interaction

At startup, app.py loads all three serialised Pipeline objects from disk using joblib. Each pipeline encapsulates both the ColumnTransformer preprocessor and the regressor, so inference requires only a single predict() call per model. The Flask server exposes five HTTP routes; the web UI communicates with these via JSON fetch requests.

3. Dataset Description

3.1 Data Generation Methodology

A synthetic dataset of 10,000 trip records was generated using numpy (random seed 42) to ensure reproducibility. The dataset was designed to mirror the structure of real ride-share trip data while incorporating the economic dynamics of the Pakistani market.

Gaussian noise (σ = PKR 150) was added to the computed fares to simulate real-world variability, and all fares were clipped to a minimum of PKR 200 to reflect base fare economics.

3.2 Feature Descriptions

#	Type	Feature	Values / Range
1	*Categorical*	**pickup**	downtown, airport, suburb, midtown, uptown
2	*Categorical*	**dropoff**	downtown, airport, suburb, midtown, uptown
3	*Numerical*	**distance**	1 – 80 km (uniform distribution)
4	*Numerical*	**passengers**	1 – 6 (integer, random)
5	*Categorical*	**timeofday**	morning, midday, evening, night, latenight
6	*Categorical*	**traffic**	low, medium, high, jam
7	*Categorical*	**ridetype**	standard, xl, premium, economy

3.3 Fare Formula

The fare is computed deterministically before noise injection:

$$fare = (200 + distance \times 80) \times traffic_multiplier \times ride_multiplier + \varepsilon$$

- Base fare: PKR 200
- Distance rate: PKR 80 per kilometre
- Traffic multiplier: 1.8× for jam conditions (low, medium, high → 1.0×)
- Ride type multiplier: 2.0× for premium (standard, xl, economy → 1.0×)
- Gaussian noise: $\varepsilon \sim N(0, 150^2)$
- Minimum fare clipping: PKR 200

3.4 Train/Test Split

The 10,000 samples were split 80/20 (train: 8,000 / test: 2,000) using sklearn's train_test_split with random_state=42 for reproducibility. The same split was applied across all three models to ensure a fair comparison.

4. Machine Learning Methodology

4.1 Preprocessing Pipeline

All three models share an identical preprocessing step implemented as a scikit-learn ColumnTransformer inside a Pipeline. This guarantees that both training and inference use the same transformation logic, preventing data leakage and ensuring consistency.

- Numerical features (distance, passengers): StandardScaler — zero mean, unit variance.
- Categorical features (pickup, dropoff, timeofday, traffic, ridetype): OneHotEncoder with handle_unknown='ignore' to gracefully handle unseen categories at inference time.

4.2 Model 1: Random Forest Regressor

Random Forest is an ensemble method that trains a large number of decision trees on bootstrapped subsets of the training data (bagging). Each tree is grown on a random subset of features at each split. Predictions are computed as the mean of all tree outputs, which substantially reduces variance compared to a single decision tree.

Parameters: n_estimators=100, random_state=42

Rationale: Random Forest is well-suited to this problem because the fare formula contains multiplicative interactions (traffic × ride type × distance) that cannot be captured by a linear model. The ensemble approach also provides robustness against the Gaussian noise in the target variable.

4.3 Model 2: Gradient Boosting Regressor

Gradient Boosting constructs an additive ensemble sequentially: each new tree is trained to minimise the residual error of the current ensemble, applying gradient descent in function space. This iterative error correction allows the model to fit complex patterns effectively.

Parameters: n_estimators=100, random_state=42

Rationale: Gradient Boosting complements Random Forest as a second tree-based method with a different bias-variance trade-off. Its sequential correction mechanism makes it particularly adept at capturing the surge pricing pattern (1.8× jam multiplier) present in the data.

4.4 Model 3: Linear Regression (Baseline)

Linear Regression fits a linear hyperplane to the feature space by minimising the sum of squared residuals (ordinary least squares). It is included as a baseline model to quantify the performance gain provided by the non-linear ensemble methods.

Parameters: None (OLS, no regularisation)

Limitation: Linear Regression cannot model the multiplicative structure of the fare formula. The 1.8× jam and 2.0× premium multipliers create interaction effects that are inherently non-linear, leading to systematic underfitting.

4.5 Evaluation Metrics

- Mean Absolute Error (MAE): Average magnitude of prediction errors in PKR. Interpretable as 'average error per trip'.

- Root Mean Square Error (RMSE): Square root of the mean squared error. Penalises large errors more heavily than MAE.
- R^2 (Coefficient of Determination): Proportion of target variance explained by the model. $R^2 = 1.0$ indicates perfect prediction.

5. Results and Model Comparison

5.1 Performance Summary

Model	MAE (PKR)	RMSE (PKR)	R^2	Confidence
🥇 **Random Forest**	**129**	**161**	**0.998**	**94%**
🥈 **Gradient Boosting**	141	187	0.998	92%
🥉 **Linear Regression**	**957**	**1,418**	0.874	74%

5.2 Analysis of Results

Both tree-based models — Random Forest and Gradient Boosting — achieve an exceptional R^2 of 0.998, explaining 99.8% of the variance in fare amounts. This demonstrates that the multiplicative fare structure is highly learnable by ensemble methods. The 12-unit MAE difference between the two (129 vs 141 PKR) is practically negligible for most use cases.

5.3 Why Random Forest Outperforms

Random Forest achieves marginally lower error than Gradient Boosting (MAE: 129 vs 141) for several reasons. Bagging-based variance reduction proves particularly effective on noisy regression targets (σ = PKR 150 noise). Additionally, Gradient Boosting's sequential nature makes it slightly more sensitive to the noise in individual predictions when n_estimators is equal.

5.4 Why Linear Regression Fails

Linear Regression produces an MAE of PKR 957 — 7.4× worse than Random Forest. The root cause is structural: the fare formula includes multiplicative interactions (traffic × ride type × distance) that a linear model cannot represent. OLS can only fit additive, linear combinations of features. The model systematically underestimates jam+premium trip fares and overestimates low-traffic economy fares.

6. Web Application

6.1 Flask Application Design

The Flask application (app.py) implements a clean REST API pattern. All three serialised Pipeline objects are loaded into a global MODELS dictionary at startup. A MODEL_META dictionary stores static metadata (MAE, RMSE, R^2, confidence, display colour) keyed by model identifier, allowing the frontend to dynamically style results without additional API calls.

6.2 Request Input Format

All POST endpoints accept a JSON body with the following fields:

- pickup (string): Origin zone — one of: downtown, airport, suburb, midtown, uptown
- dropoff (string): Destination zone — same value set as pickup
- distance (float): Trip distance in kilometres (1.0 – 80.0)
- passengers (int): Number of passengers (1 – 6)
- timeofday (string): Time segment — morning, midday, evening, night, latenight
- traffic (string): Traffic level — low, medium, high, jam
- ridetype (string): Service tier — standard, xl, premium, economy
- model (string, optional): Model key for /predict — random_forest, gradient_boosting, linear_regression

6.3 Surge Pricing Logic

Surge pricing is detected server-side: when the traffic field is 'high' or 'jam', the API appends surge: '1.5x' to the response. This indicator is rendered visually in the UI as an amber warning badge, alerting users to elevated fare conditions before they book.

7. Discussion

7.1 Key Findings

The primary finding of this project is that tree-based ensemble methods are ideally suited for structured, rule-based fare prediction tasks. The multiplicative fare structure creates decision boundaries that are easily approximated by axis-aligned decision splits, explaining why both Random Forest and Gradient Boosting achieve near-perfect R^2 values.

The dramatic underperformance of Linear Regression (MAE 7.4× higher) is an important pedagogical result: it demonstrates that model selection must be informed by the structural properties of the target variable, not by simplicity alone.

7.2 Limitations

- Synthetic data: The dataset was generated algorithmically and does not incorporate real-world complexities such as route geometry, driver availability, or time-varying demand.

- Zone-based features: Pickup and dropoff locations are represented as categorical zones rather than GPS coordinates, limiting spatial granularity.
- No real-world validation: Models have not been evaluated against actual Uber or Careem trip data from the Pakistani market.
- Fixed multipliers: The 1.8× jam and 2.0× premium multipliers are hard-coded constants; real surge pricing is dynamic and market-driven.
- No temporal features: Date, day of week, and seasonal patterns are not represented in the dataset.

7.3 Comparison with Related Work

Fare prediction research on the New York City Taxi dataset has consistently shown that gradient-boosted trees outperform linear models, corroborating the findings of this project. Studies by Jain et al. (2018) and others have demonstrated R^2 values in the 0.96–0.99 range on real-world taxi data, suggesting that the $R^2 = 0.998$ achieved here is achievable in production with appropriate feature engineering.

8. Conclusion and Future Work

8.1 Summary

RideFare AI successfully delivers an AI-powered taxi fare prediction platform for the Pakistani ride-share market. Three machine learning models were trained, evaluated, and deployed behind a Flask REST API with a production-quality web interface. The Random Forest model achieves near-perfect prediction accuracy ($R^2 = 0.998$, MAE = PKR 129), making it the clear production choice.

The project demonstrates the complete machine learning engineering lifecycle: data generation, preprocessing pipeline construction, multi-model training and evaluation, model serialisation, REST API development, and frontend integration.

8.2 Future Work

The following enhancements are planned for future iterations:

- Real Data Integration: Replace synthetic data with real trip records from Uber/Careem Pakistan APIs or open-source datasets.
- GPS Coordinates: Replace categorical zone features with latitude/longitude pairs and route distance calculation via Google Maps API.
- Deep Learning: Evaluate XGBoost, LightGBM, and neural network regressors for marginal accuracy improvements.
- Time-Series Surge Prediction: Train a separate model to forecast surge multipliers based on historical time-of-day and day-of-week patterns.
- Mobile Application: Develop a React Native or Flutter mobile app consuming the Flask REST API.
- Real-Time Traffic API: Integrate Google Maps or HERE Traffic API for live traffic-aware fare adjustments.
- Continuous Retraining: Implement a user feedback loop where actual trip fares are collected to retrain models incrementally.
- Multi-City Support: Extend coverage to Karachi, Lahore, and Islamabad with city-specific fare multipliers.